

The Value-Typing Paradox

Why `color: --ink` was refused, and what the escape lists are hiding

Spacetime architecture note · FUP-186 · 2026-08-13

The symptom

Spacetime prefers `color: --ink` over `color: var(--ink)`. The reasoning is sound: `--name` already names a value unambiguously, so `var()` is CSS ceremony the language does not need.

The compiler refused it:

```
error[E0958]: `--ink` is not a valid color for `color`
--> page.st:8:10
|
8 |   color: --ink;
|           ^^^^^
```

The rule this violates was settled months earlier, in PLAN-136 W2: **a token reference is a value in every value position**. The function that answers that question, `capture_type_accepts`, has honoured it since.

So why did a different part of the compiler disagree?

The immediate cause

The rule had **two** enforcement sites and **one** guard. `directive_property_diagnostics` (E0967) asks `is_token_reference_value` before refusing. `validate_declaration_value` (E0958) never did. Same rule, two implementations, one of them wrong.

That is a one-line fix, and it is done. But it is the third time this arc has found the same shape, and the fix I applied is itself the problem worth writing down.

The fix that is also the problem

Here is what `validate_declaration_value` looks like now, at the point where it decides whether a value is even **its business**:

```
// Not CSS at all: a Spacetime binding, hole, element ref, or an
// animation range (`0 -> 1`).
if value.contains('$')
  || value.contains('`')
  || value.contains('&')
  || value.contains("->")
  || value.starts_with('%')
  || value.starts_with('@')
{
  return None;
```

```

}

// A token reference is a value in every value position.
if crate::syntax::events::is_token_reference(value) {
    return None;
}

// CSS-wide keywords are legal on every property.
if matches!(folded.as_str(),
    "inherit" | "initial" | "unset" | "revert" | "revert-layer") {
    return None;
}

```

Four escape hatches, each added when something legal got refused. Each one is correct. Together they are a hand-maintained list of “**things that are not CSS**,” which is exactly the kind of list this codebase has spent nine commits deleting elsewhere.

And it is not one list. Fifty-five sites across the compiler independently ask some version of “**is this Spacetime syntax rather than a CSS value?**” — the knowledge is smeared across the codebase, and every one of them can be wrong in a different direction.

The paradox, stated plainly

A CSS validator must decide whether a value is **well-typed**. But it can only do that for values that are **CSS at all** — and in Spacetime, most values in a declaration are not. So before it can type-check anything, it must first answer a question it has no principled way to answer, and it answers it with a list of sigils someone remembered to add.

Three type systems that do not know about each other

The refusal is a symptom of a deeper structure. Spacetime has **three** systems that each answer “what kind of value is this?”, and none of them is the source of truth for the others.

1. Scalar types — `stdlib/scalars/types.st`

Fourteen rows. Each one says what a scalar **means**:

```

%scalar_type color {
    %capture "color"
    %schema "string"
    %format "color"
    %zero ""
    %widget "color"
    %docs "A CSS colour: hex, colour function, or keyword."
}

```

This is a semantic table: how the value serialises, what editor widget it gets, what its empty value is. It is **data**, in the `stdlib`, and four Rust maps read it. It knows nothing about grammar.

2. Capture types — `stdlib/capture-types/*.st`

Eighty-two productions. Each one says what a value **looks like**:

```
%capture_type variant_payload {  
    ( $ty:typeref ", "? ) *  
}
```

This is an executable grammar: a parser, written in Spacetime, that either matches a token run or does not. It knows nothing about meaning — `color` as a capture type can tell you `#ff6b47` matches and `"#ff6b47"` does not, but not that a colour is serialised as a string or edited with a colour picker.

3. CSS property validation — `lightningcss`, via `src/validation/css.rs`

A vendored CSS parser that knows the real specification: that `color` takes a `<color>`, that `grid-template-areas` takes strings, that `1px solid --ink` is a shorthand with three components.

It is the only one of the three that knows what a **property** expects. It is also the only one that has never heard of Spacetime.

	knows meaning	knows shape	knows the property
scalar types	yes	no	no
capture types	no	yes	no
lightningcss	no	yes, for CSS	yes

Nothing in the table has all three columns. The escape lists are the seam where they meet, and a seam maintained by hand is a seam that drifts.

Why the obvious fixes do not work

“Just make `lightningcss` aware of Spacetime”

It is vendored, and the AGENTS rule is explicit: **vendor and own behind a primitive, do not hand-roll**. Patching a third-party CSS parser to understand `$binding` and `--token` forks it permanently, and every upgrade becomes a merge.

“Pre-substitute Spacetime values, then validate the CSS”

Tempting, and wrong in a way that matters: the substituted value is not knowable at build time. `var(--ink)` may be defined in a project prelude, a `@tokens` block, or plain CSS the compiler never sees. A validator that demands resolution before it will judge shape would refuse every legitimate deferral — and refusing a legal build is worse than missing a diagnostic.

“Type the property, then check the value against that type”

This is what E0967 does, and it works — for **directive** properties, where the `%form` declares `easing: $easing:easing` and the type is known. It does not generalise to plain CSS declarations, because nothing declares the type of `color` in Spacetime. That knowledge lives only in `lightningcss`.

“Widen the escape list”

The current answer. It works until the next legal spelling arrives, at which point someone hits a false refusal, and the fix is another `||`.

The cost of getting this wrong is asymmetric

A missed diagnostic is a diagnostic that fires later, or never. A **false refusal blocks a build the author cannot fix** — they wrote legal Spacetime and the compiler said no. This asymmetry is why the escape lists keep growing: every one of them was added under pressure, correctly, to unblock someone.

The structural fix

The escape lists exist because the question is being asked at the wrong moment, by a component that lacks the information to answer it.

By the time `validate_declaration_value` sees `--ink`, it has a **string**. It must reverse-engineer what that string is by inspecting its characters. That is the whole defect: a lexical guess, standing in for knowledge the compiler had earlier and threw away.

The proposal: a value is a tagged node, not a string

When the parser reads a declaration value, it already knows what it is looking at. `$binding` is a binding because it lexed a `$`. `--ink` is a token reference because it lexed two dashes and an identifier. `hole` is a hole. `0 -> 1` is a range because it lexed an arrow.

That knowledge should survive into the value, rather than being recovered from its spelling later:

```
enum ValueNode {
    Css(String),           // genuinely CSS text – validate it
    TokenRef(String),      // --ink – a value, resolved by the cascade
    Binding(String),       // $sig – a Spacetime signal
    Hole(String),          // `expr` – an interpolation
    ElementRef(String),    // &self – an element
    Range(Box<ValueNode>, Box<ValueNode>), // 0 -> 1
    Compound(Vec<ValueNode>), // lpx solid --ink
}
```

Then the validator’s first question stops being “**does this string contain a dollar sign?**” and becomes “**is this node `Css`?**” — which is not a guess. Every other variant is not CSS by construction, and no list is needed to say so.

What this buys, concretely

Escape lists disappear The four hatches collapse into one match arm. The fifty-five scattered sigil checks have a single node kind to ask instead.

Compound values work `border: lpx solid --ink` is a `Compound` of three nodes. The CSS parts can be validated as CSS; the `TokenRef` is skipped. Today this case is unhandled — the current lowering only recognises a value that is **entirely** one reference, because a string rewrite cannot see inside a shorthand without re-parsing it.

Lowering becomes trivial and safe `TokenRef("--ink")` emits `var(--ink)`. No regex, no risk of rewriting `content: "--not-a-token"`, and no risk of wrapping a custom property's own declaration into `--ink: var(#222)`.

The three type systems get a shared vocabulary a `%scalar_type` describes what a `ValueNode` means; a `%capture_type` describes which token runs **produce** one; lightningcss judges only the `Css` variant. Each keeps its own job, and the node is the contract between them.

The honest costs

This is not free, and the plan should say so.

1. **It is a parser change, not a validator change.** The CSS value parser must produce nodes instead of strings, and every consumer of a value string must learn the new type. There are twenty-one `CssExpr::Raw` construction sites that currently pass CSS through as opaque text.
2. **Raw cannot simply be deleted.** Some CSS genuinely is opaque to us — at-rule preludes, vendor syntax, things lightningcss round-trips. `Raw` must survive as a variant, which means the discipline is “prefer a typed node, fall back to `Raw`” rather than “no strings anywhere.”
3. **It touches the emitter.** Two declaration emitters exist (`emit_decl` and `emit_decl_to_writer`, the source-map path). Both must agree, or a bare token works in a normal build and breaks under `--sourcemap`.
4. **Migration is incremental but wide.** The value is that each converted site deletes an escape check; the cost is that the benefit only fully lands when the last one is converted.

Where this leaves FUP-186

The immediate work splits cleanly along the argument above:

step	what	status
1	Admit <code>--ink</code> in <code>validate_declaration_value</code> — the missing guard	done, gated
2	Lower a whole-value <code>TokenRef</code> to <code>var()</code> at both emitters	done for the paths that use them
3	Reach the values that bypass those emitters (<code>CssExpr::Raw</code>)	open
4	Compound values (<code>1px solid --ink</code>)	open — blocked on the node
5	The <code>ValueNode</code> refactor proper	proposed here

Steps 3 and 4 are the fork. Step 3 can be done with a string rewrite over `Raw` text, and it would work today for whole-value references. Step 4 cannot: a regex over CSS values is how `content: "--not-a-token"` becomes garbage.

The recommendation

Do not ship a string-rewriting lowering for compound values. Do step 3 only if the `ValueNode` work is genuinely deferred; otherwise the string rewrite is a fifth escape hatch, and it will have to be deleted along with the other four.

The general lesson

Three defects this week shared one shape: a construct that compiled cleanly while doing nothing. `%when $visible` could never see a runtime signal. `const el = el;` threw at load. `%states` ' unconditional branch was never written.

This is the fourth, and it is the same shape inverted — a construct that refuses something legal, cleanly, for a reason no test could catch, because the test would have to know the list too.

In both directions the cause is the same: **knowledge that existed at one stage of the pipeline, discarded, and then guessed at downstream.** The parser knew `--ink` was a token reference. By the time anyone needed that fact, all that remained was a string and a list of characters to look for.